

Despojados: <https://judge.beecrowd.com/pt/problems/view/2661>

Geralmente pensamos em força bruta para resolver os problemas, mas nesse caso é inviável

Pseudo código da força bruta:

```
divisores = acha_divisores(N)

resposta = 0

for div in divisores:
    fatores_primos = fatorar(div)

    se (nao_tem_repetidos) e (quantidade_de_primos >= 2):
        resposta += 1

print(resposta)
```

acha_divisores: No pior caso roda $10^6 (\sqrt{10^{12}})$

for div in divisores: O laço roda D vezes. Para um número na casa dos 10^{12} , o máximo de divisores que ele pode ter é cerca de 6.720

fatorar(div): Dentro do laço, fatorar o maior divisor exige, no pior caso, mais 10^6 operações ($\sqrt{10^{12}}$).

Total de operações: $10^6 + (6.720 * 10^6) = 6.721 * 10^6$

Limite por segundo em python: $\sim 10^7$ operações

Primeiro vamos definir como é formado o divisor de um número:

Qualquer divisor de um número é formado por uma combinação de seus fatores primos

Exemplo: $N = 12$ (Fatores primos: 2, 2, 3)

- O divisor 6 é a combinação: $2 * 3$
- O divisor 4 é a combinação: $2 * 2$
- O divisor 12 é a combinação: $2 * 2 * 3$

Agora fica mais fácil de ver como podemos resolver: precisamos ver quantas dessas combinações são *despojados*

Para ficar mais fácil, podemos remover os fatores repetidos de N :

Exemplo: $N = 12 \rightarrow$ Fatores primos: 2, 2, 3

Removendo os fatores repetidos, temos a lista final: 2,3

Agora todas as combinações desses primos terão números primos distintos em sua composição

Por exemplo, para $N = 252$, temos os fatores primos (sem repetições): 2, 3 e 7

Para saber todas as combinações, precisamos fazer:

- Combinação de 3 para escolher 1
- Combinação de 3 para escolher 2
- Combinação de 3 para escolher 3

Mas não precisamos computar tudo isso na mão, a fórmula de todas as combinações é: $2^K - 1$

Só existe mais um detalhe, o exercício define *despojados* sendo os números que possuem **2 ou mais** primos distintos em sua decomposição

Então precisamos remover as combinações que possuem apenas 1 primo em sua decomposição:

- Quantas combinações existem com apenas 1 primo?

Resposta: Exatamente K combinações. O 2 sozinho, o 3 sozinho, etc

Então nossa fórmula final é: **$2^K - 1 - K$**

Código:

```
import math

n = int(input())
K = 0

for i in range(2, math.isqrt(n) + 1):
    if n % i == 0:
        while n % i == 0:
            n //= i
            K += 1

if n > 1:
    K += 1

print(2 ** K - 1 - K)
```